

Batch Size at 320×128

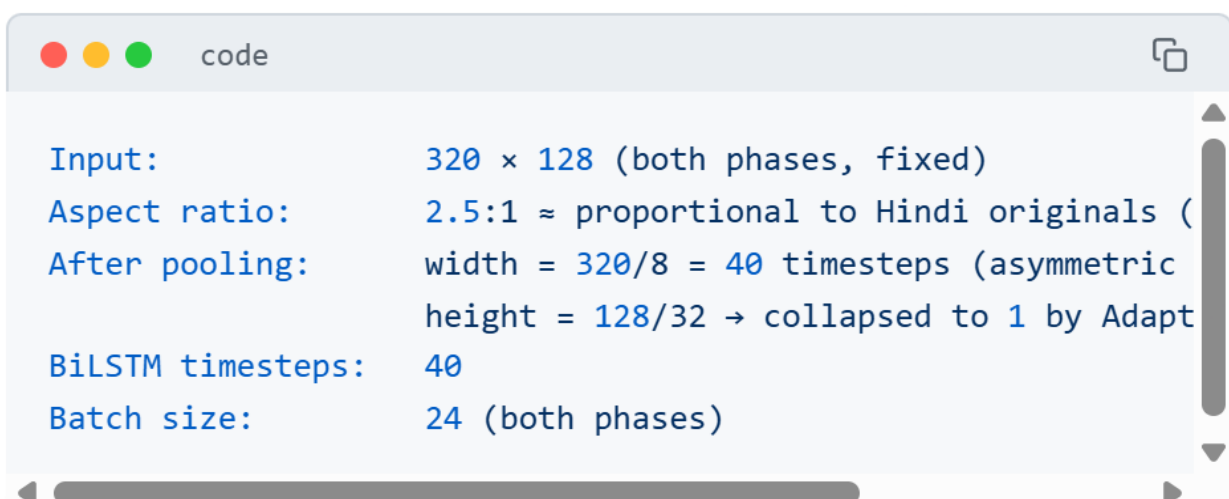
Yes, meaningfully better. Memory scales with spatial dimensions of feature maps, the bottleneck being block 1 which is still at full input resolution.

At 512×128: roughly $512 \times 128 \times 64$ feature values per image → safe at batch 16
At 320×128: roughly $320 \times 128 \times 64$ feature values per image → safe at **batch 24**

That's 50% more samples per gradient update, which directly recovers the gradient noise concern from earlier. Batch 24 is a solid number — stable gradients, fits comfortably in 16GB T4 VRAM, and leaves headroom for the BiLSTM's sequence operations.

Finalized Complete Design

Resolution and Core Numbers



```
Input:          320 × 128 (both phases, fixed)
Aspect ratio:   2.5:1 ≈ proportional to Hindi originals (
After pooling:  width = 320/8 = 40 timesteps (asymmetric
                  height = 128/32 → collapsed to 1 by Adapt
BiLSTM timesteps: 40
Batch size:     24 (both phases)
```

Model Architecture — Full Stack

code



INPUT

Image: 320 × 128, grayscale converted to 3-channel

VGG16 BLOCK 1 (layers 0-4)

Conv2d(3 → 64, 3×3) → ReLU

Conv2d(64 → 64, 3×3) → ReLU

MaxPool2d(2,2) → [B, 64, 64, 160]

Params: ~38K

VGG16 BLOCK 2 (layers 5-9)

Conv2d(64 → 128, 3×3) → ReLU

Conv2d(128 → 128, 3×3) → ReLU

MaxPool2d(2,2) → [B, 128, 32, 80]

Params: ~221K

VGG16 BLOCK 3 (layers 10-16)

Conv2d(128 → 256, 3×3) → ReLU

Conv2d(256 → 256, 3×3) → ReLU

Conv2d(256 → 256, 3×3) → ReLU

MaxPool2d(2,2) → [B, 256, 16, 40]

Params: ~1.47M

VGG16 BLOCK 4 (layers 17-23) ← ASYMMETRIC POOL

Conv2d(256 → 512, 3×3) → ReLU (layer 17)

Conv2d(512 → 512, 3×3) → ReLU (layer 19)

Conv2d(512 → 512, 3×3) → ReLU (layer 21)

MaxPool2d(2,1) → [B, 512, 8, 40] ← height hal

Params: ~5.9M

VGG16 BLOCK 5 (layers 24-30) ← ASYMMETRIC POOL

Conv2d(512 → 512, 3×3) → ReLU (layer 24)

Conv2d(512 → 512, 3×3) → ReLU (layer 26)

Conv2d(512 → 512, 3×3) → ReLU (layer 28)

`MaxPool2d(2,1)` → [B, 512, 4, 40] ← height halved

Params: ~7.08M

BRIDGE

`AdaptiveAvgPool2d((1, None))` → [B, 512, 1, 40]

Squeeze + Permute → [40, B, 512]

BILSTM

`BiLSTM(input=512, hidden=256, layers=2, bidirectional=True)`

dropout=0.3 (Phase 1) / 0.4 (Phase 2) between layers

Output: [40, B, 512] (256 × 2 directions)

Params: ~3.15M

FC HEAD

`Dropout(0.3 / 0.4)`

`Linear(512 → num_classes)`

`LogSoftmax(dim=2)`

Output: [40, B, num_classes]

CTC

Training: `nn.CTCLoss(blank=0, reduction='mean', zero_infinity_value=-1000000)`

Inference: greedy decode → argmax → collapse repeats → drop

Freezing Strategy

Block	Layers	Phase 1 (92k Hindi)	Phase 2 (24k Nepali)	Reasoning
Block 1	0–4			Pure edge detectors, ImageNet is fine
				Simple

Block 2	5–9			Simple curves, nothing script-specific
Block 3	10–16			Phase 1 learns Devanagari stroke junctions. Phase 2 protects them from 24k font bias
Block 4 conv 1-2	17–20			Phase 1 learns character parts. Phase 2 freezes to preserve handwriting robustness
Block 4 conv 3	21–23		 (LR 1e-6)	Highest block 4 abstraction, slight Nepali adaptation
				Full glyphs,

Block 5	24–30		 (LR 1e-6)	ligatures — needs Nepali vocabulary nudge
BiLSTM	—	 (LR	 (LR 1e-4)	Sequence learning, always

Trainable parameter count by phase:

- Phase 1: ~17.6M trainable (blocks 3–5 + BiLSTM + FC)
- Phase 2: ~10.3M trainable (block 4 tail + block 5 + BiLSTM + FC)

Training Configuration

Phase 1 — Hindi Handwriting

```
code
```

```

Dataset:          92k images (train ~60k / val ~15k / test ~17k)
Resolution:       320 × 128
Batch size:       24
Optimizer:        AdamW
LR:               1e-4 flat across all trainable layers
LR scheduler:     ReduceLROnPlateau(patience=3, factor=0.5)
Max epochs:       30
Early stopping:   patience=5, min_delta=1e-4 on val CTC loss
Checkpoint:       every epoch → phase1_epoch{N:03d}_acc{acc:.4f}
Augmentation:     rotation ±5°, gaussian blur, random brightness,
                  slight elastic distortion (Devanagari-safe)
BiLSTM dropout:   0.3
FC dropout:       0.3

```

Phase 2 — Nepali Font Images

```
Dataset:          24k images (train ~18k / val ~4k / test ~2k)
Resolution:       320 × 128 (same as Phase 1 – no timestep di
Batch size:       24
Optimizer:        AdamW
LR (CNN tail):    1e-6    (block 4 conv 3 + block 5)
LR (BiLSTM+FC):   1e-4
LR scheduler:     ReduceLROnPlateau(patience=3, factor=0.5)
Max epochs:       20
Early stopping:   patience=5, min_delta=1e-4 on val CTC loss
Checkpoint:       every epoch → phase2_epoch{N:03d}_acc{acc:.
Augmentation:     random brightness/contrast only – fonts are
                  heavy augmentation would create unrealistic

BiLSTM dropout:   0.4
FC dropout:       0.4
Weights loaded:   best Phase 1 checkpoint (auto-selected by 1
```

Vocabulary and Character Set

```
Phase 1 vocab:     built from all unique characters in Hindi t
                  includes full Devanagari Unicode range from
                  blank token = index 0

Phase 2 vocab:     Phase 1 vocab EXTENDED with any new chars f
                  never replaced, only appended – CTC alignme
                  NLP vocab.json: 789 first names + 485 last
                  = 1,353 total entries for postprocessor
```

Data Pipeline

code

```
Both datasets → list of (absolute_image_path, label_string) t
↓
DevanagariDataset(pairs, transform, char2idx)
↓
DataLoader(batch_size=24, num_workers=4, pin_me

Collate function handles variable-length label sequences for
Image transform: resize → grayscale → expand to 3ch → normali
ImageNet mean/std for Phase 1 (VGG16 expects
same normalization Phase 2 for consistency
```

Metrics — Full Definition

Metric	Computed	Formula
CTC Loss	Every epoch, train+val	Negative log-likelihood over all valid alignments
CER	Every epoch val, final test	$\text{edit_distance}(\text{pred}, \text{truth}) / \text{len}(\text{truth})$, mean over set
Word Accuracy	Every epoch val, final test	$\text{exact_matches} / \text{total_samples}$
WER	Final test	$1 - \text{Word Accuracy}$ (redundant with above)

	only	but standard to report)
Norm. Edit Distance	Final test only	CER capped at 1.0 per sample, then averaged
Confusion	Test set only	per-character, top 20

Graphs — Six Total

```

code
Phase 1:
  G1 – Train CTC loss vs Val CTC loss, per epoch
  G2 – Val CER per epoch
  G3 – Val Word Accuracy per epoch

Phase 2:
  G4 – Train CTC loss vs Val CTC loss, per epoch
  G5 – Val CER per epoch
  G6 – Val Word Accuracy per epoch

Report additions:
  G7 – Bar chart: Phase 1 vs Phase 2 Word Accuracy on Nepali
        (transfer learning evidence)
  G8 – Character confusion matrix heatmap, test set

```

Phase 3 — NLP Postprocessor

```

code
Input:    raw CRNN output string
Process:  1. Check if output exists in vocab.json exactly → p
          2. If not found → Levenshtein search within edit di

```


3. If field type known (name/district) → constrain
district: 79 candidates only
name: 1,274 candidates (789+485)
4. If no match within distance 2 → return raw output

Output: corrected string

Complete Parameter Count Summary

code

```
VGG16 features (total):           ~14.7M
BiLSTM (2 layers, bidirectional): ~3.15M
FC head:                          ~50-80K (depends on vocab size)
-----
Total model:                      ~18M parameters

Phase 1 trainable:                ~17.6M
Phase 2 trainable:                ~10.3M
Phase 2 frozen:                   ~7.4M
```

That's the complete locked design. Everything is consistent, reasoned, and ready to implement. Start the notebook?